- Depending on the result size from the primary key query, filtering may be slow. Filters work without an index: every item must be inspected. Imagine you have stock prices in DynamoDB, with a partition key and sort key: the partition key is a ticker like AAPL, and the sort key is a timestamp. You can make a query to retrieve all stock prices of Apple (AAPL) between two timestamps (20100101 and 20150101). But if you want to return prices only on Mondays, you need to filter over all prices to return only 20% (one out of five trading days each week) of them. That's wasting a lot of resources!
- You can only query the primary key. Returning a list of all tasks that belong to a certain category for all users isn't possible, because you can't query the `category` attribute.

You can solve these problems with secondary indexes. Let's look at how they work.

12.4.3 Using global secondary indexes for more flexible queries

In this section, you will create a secondary index that allows you to query the tasks belonging to a certain category.

A *global secondary index* is a projection of your original table that is automatically maintained by DynamoDB. Items in an index don't have a primary key, just a key. This key is not necessarily unique within the index. Imagine a table of users where each user has a country attribute. You then create a global secondary index where the country is the new partition key. As you can see, many users can live in the same country, so that key is not unique in the index.